JavaScript

Events, DOM, Workers

О спикере

Максим Гатилин

- 11 лет в разработке
- В Школе Программистов - второй год
- В HH занимаюсь звонками между работодателями и соискателями

Events

Event

События в JavaScript — это сигналы, которые сообщают коду о различных действиях, происходящих в браузере или на веб-странице.

Эти действия могут быть инициированы пользователем (например, клики мышью, нажатия клавиш) или браузером (загрузка страницы, изменение размера окна).

EventTarget

Объекты, которые могут генерировать события и могут иметь подписчиков на эти события

Примеры — window, document, Element

Методы:

- addEventListener — добавляет обработчик события
- removeEventListener — снимает обработчик
- dispatchEvent — генерирует событие на объекте

Обработчики событий можно назначать через свойства .on* (onclick)

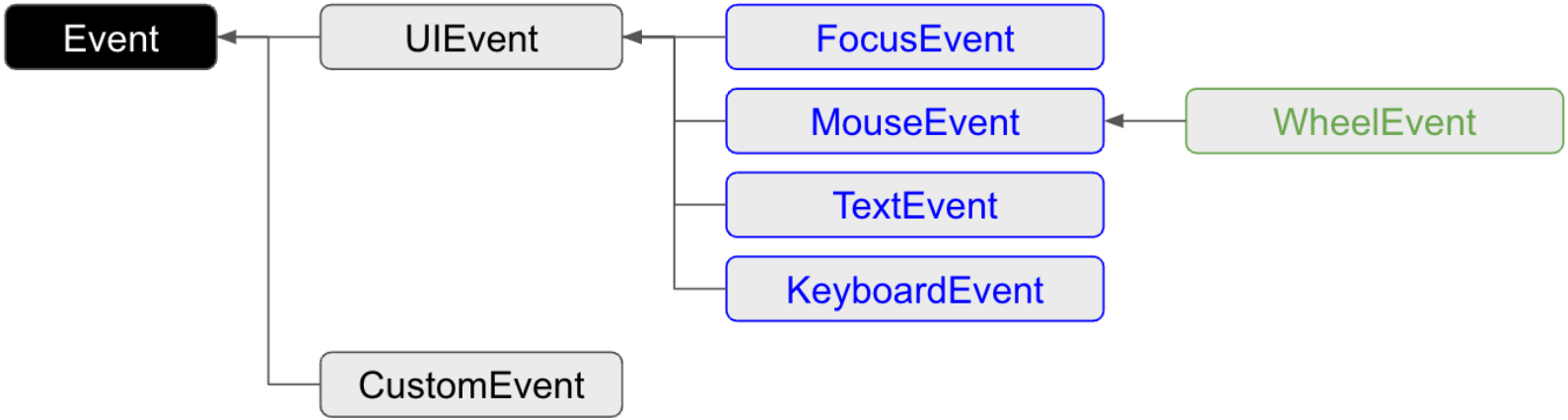
Вызовем событие на document и поймаем его обработчиком

Пример

Event

Базовый класс всех событий происходящих в DOM

Объект события приходит первым аргументом в обработчик



event.isTrusted

Если событие создано непосредственно действием пользователя, флаг isTrusted имеет значение true

Если же событие создано или вызвано с помощью JavaScript — значением флага будет false

IE не поддерживает это свойство

Посмотрим на значение флага isTrusted у событий вызванных разными способами

Пример

event.preventDefault()

С помощью event.preventDefault() можно отменить штатное поведение браузера

Примеры использования:

- Отмена перехода по ссылке
- Игнорирование вводимого символа в текстовое поле
- Отмена поведения чекбокса / радио-кнопки

Отменим программно поведение по умолчанию на ссылке, текстовом поле и чекбоксе

Пример

Определение зажатия системных клавиш

У событий класса Touch/Mouse/Keyboard Event есть дополнительные флаги для определения зажатых системных клавиш

- event.altKey — зажата ли клавиша ALT
- event.metaKey — зажата ли клавиша WIN / COMMAND
- event.shiftKey — зажат ли SHIFT
- event.ctrlKey — зажат ли CTRL

Определим зажатие системных клавиш при вводе текста в textarea

Пример

event.target / event.currentTarget

Для того чтобы определить элемент на котором событие непосредственно произошло используется свойство event.target

Для определения элемента на котором установлен обработчик используется event.currentTarget

event.target и event.currentTarget совпадают если обработчик установлен непосредственно на элемент на котором вызывается событие

Сравним значения

event.target и

event.currentTarget

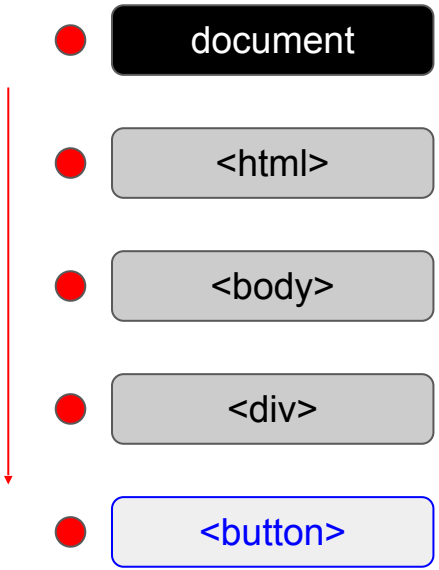
Пример

Перехват события (capture)

При клике на элемент браузер пройдет от вершины дерева до целевого элемента — эта фаза называется “перехват” или “погружение”

Обработчики событий созданные через свойства element.on* не вызываются на этой стадии

Если требуется обработать событие на этапе перехвата можно использовать метод addEventListener передав третьим аргументом true



Обработаем событие click на этапе погружения

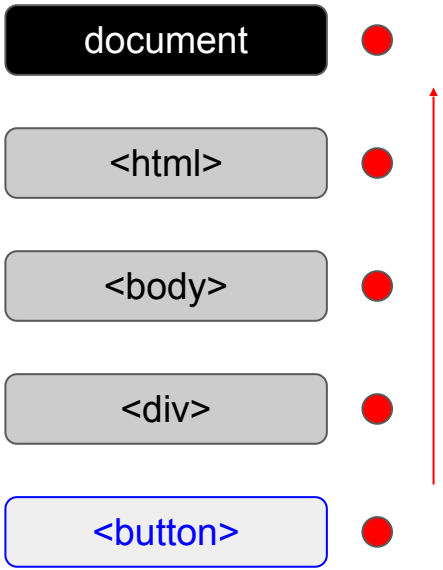
Пример

Всплытие события (bubbling)

Как только достигнут целевой элемент браузер начнет движение вверх по дереву, эта фаза называется “всплытие”

По умолчанию все обработчики устанавливаются именно на этап всплытия

Некоторые события не всплывают, например focus



Обработаем событие click на этапе всплытия

Пример

event.stopPropagation()

С помощью event.stopPropagation() можно остановить погружение или всплытие, в зависимости от того на какую фазу установлен обработчик

Для остановки выполнения других обработчиков событий на текущем элементе используется метод event.stopImmediatePropagation()

По возможности лучше избегать использования этих методов

Внезапно вам может потребоваться отлавливать все события на document, например для аналитики

Остановим всплытие и погружение события click

Пример

Создание события с помощью скрипта

Для создания экземпляра события используется конструктор new Event()

Первым аргументом указывается название события — может быть любым строковым значением

Вторым аргументом передаются параметры события

Все события поддерживают следующие параметры:

- bubbles — всплывает ли событие
- cancelable — возможно ли отменить действие по умолчанию

CustomEvent vs Event

CustomEvent является дочерним классом Event и позволяет задать при создании поле detail

Значение detail может быть любым

Для внутренних событий правильней использовать именно CustomEvent

Создадим и вызовем

CustomEvent

Пример

MouseEvent

Для обработки событий мыши используется подкласс MouseEvent

- mouseover / mouseout — вызывается когда курсор заходит / уходит с элемента. Поддерживает всплытие / погружение, делегируется дочерним элементам
- mouseenter / mouseleave — похоже на предыдущее, но не всплывает и не погружается
- mousedown / mouseup — нажатие и отпускание кнопки мыши
- mousemove — движение курсора мыши

Поставим обработчики событий мыши и посмотрим когда они вызываются

Пример

Что интересного можно узнать о событии мыши

- button — нажатая кнопка мыши
- clientX / clientY — координаты внутри окна браузера
- offsetX / offsetY — внутри элемента на котором вызывается событие
- pageX / pageY — координаты на странице вне зависимости от скролла
- screenX / screenY — координаты внутри экрана

Посмотрим чем отличаются различные типы координат в MouseEvent

Пример

TouchEvents

Для обработки тач-событий есть подкласс TouchEvent

- touchstart / touchend
- touchmove
- touchcancel

В самом событии мы можем получить:

- targetTouches — касания на элементе на котором установлен обработчик
- touches — касания на всём документе

Вызовем TouchEvent и посмотрим на подробные данные

Пример

PointerEvents

Этот класс объединяет в себе перехват событий мыши и тач-устройств

- pointerover / pointerout
- pointerenter / pointerleave
- pointerdown / pointerup
- pointermove

Если на элементе есть обработчики MouseEvent и PointerEvent — будут вызваны оба

Чтобы этого избежать в обработчике PointerEvent нужно вызвать event.preventDefault()

Готов ответить на ваши вопросы

DOM API

Node

Интерфейс описывающий DOM-узел, наследует EventTarget

Примеры — Element, document, Text, Attr

Основные свойства: childNodes, firstChild, lastChild, nextSibling, previousSibling, nodeType, parentNode, parentElement

Основные методы: appendChild, removeChild, cloneNode

Не все объекты наследующие Node могут иметь дочерние элементы, например HTML атрибут и текстовая нода при попытке вызова appendChild вызовут исключение

Посмотрим на то как работают Node

Пример

Миксин ParentNode

Миксин — это набор свойств / методов расширяющий интерфейс. Не может выступать в роли самостоятельного конструктора

Миксин ParentNode добавляет в Node возможности для работы с дочерними элементами

Добавляет свойства: children, childElementCount, firstElementChild, lastElementChild

Реализует методы: append, prepend, querySelector, querySelectorAll, replaceChildren

Миксин ChildNode

Добавляет в Node методы:

- remove — удаление ноды
- before — добавление ноды перед текущей
- after — добавление ноды после текущей
- replaceWith — замена текущей ноды на переданную

Element

Наследует Node с миксинами ParentNode и ChildNode, является базовым интерфейсом для всех HTML и SVG элементов

Конкретные элементы (div,span,a,input) добавляют свой функционал

Посмотрим на различия

Node и Element

Пример

Готов ответить на ваши вопросы

IntersectionObserver

Позволяет создать наблюдателя за пересечением объектов

Конструктор new IntersectionObserver(callback[, options])

С помощью опции root можно задать объект относительно которого будет проводится проверка

observer.observe(element) начнет отслеживать пересечение заданного элемента с заданным root (по умолчанию — viewport документа)

Перестать отслеживать можно с помощью метода unobserve

Не работает в IE

Поймаем событие пересечения элемента и viewPort браузера

Пример

Workers

Web Worker

Позволяет создать параллельный поток выполнения JavaScript

Существует с момента создания до закрытия страницы или программного уничтожения

Общение с основным сайтом осуществляется посредством postMessage

Worker не может манипулировать DOM, ему недоступны Storage

Доступны WebSocket, XMLHttpRequest, Cache API, можно создавать новые Worker’ы, в ограниченном виде доступен Canvas

Сценарии применения Web Worker

- В воркерах можно выполнять сложную работу не нагружая основной процесс, например подсветку синтаксиса в онлайн редакторе или вычисление различий в тексте
- Также туда можно перенести сложные манипуляции с изображениями (включая WebGL) посредством OffscreenCanvas
- Воркеры могут осуществлять AJAX-запросы или заниматься обработкой WebSocket-соединения

Создадим простой Worker

Пример

ServiceWorker

Наследует все свойства от Web Worker

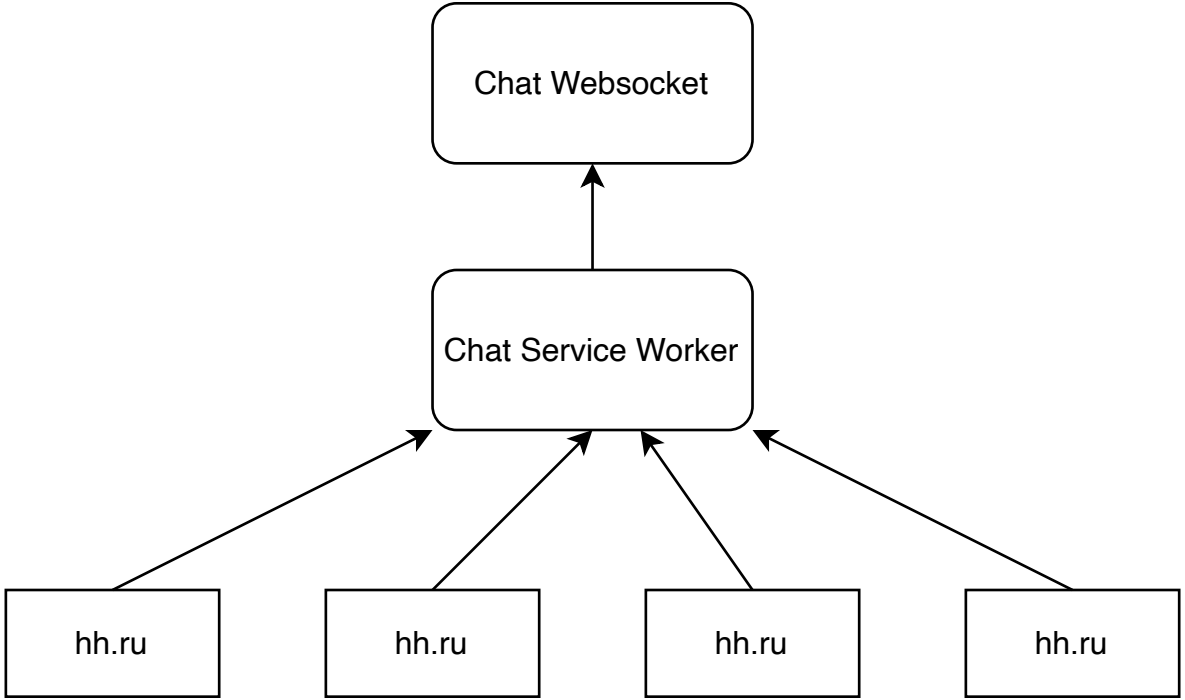
Привязывается к конкретному origin страницы на которой был инициализирован

Выступает в роли кэширующей прослойки

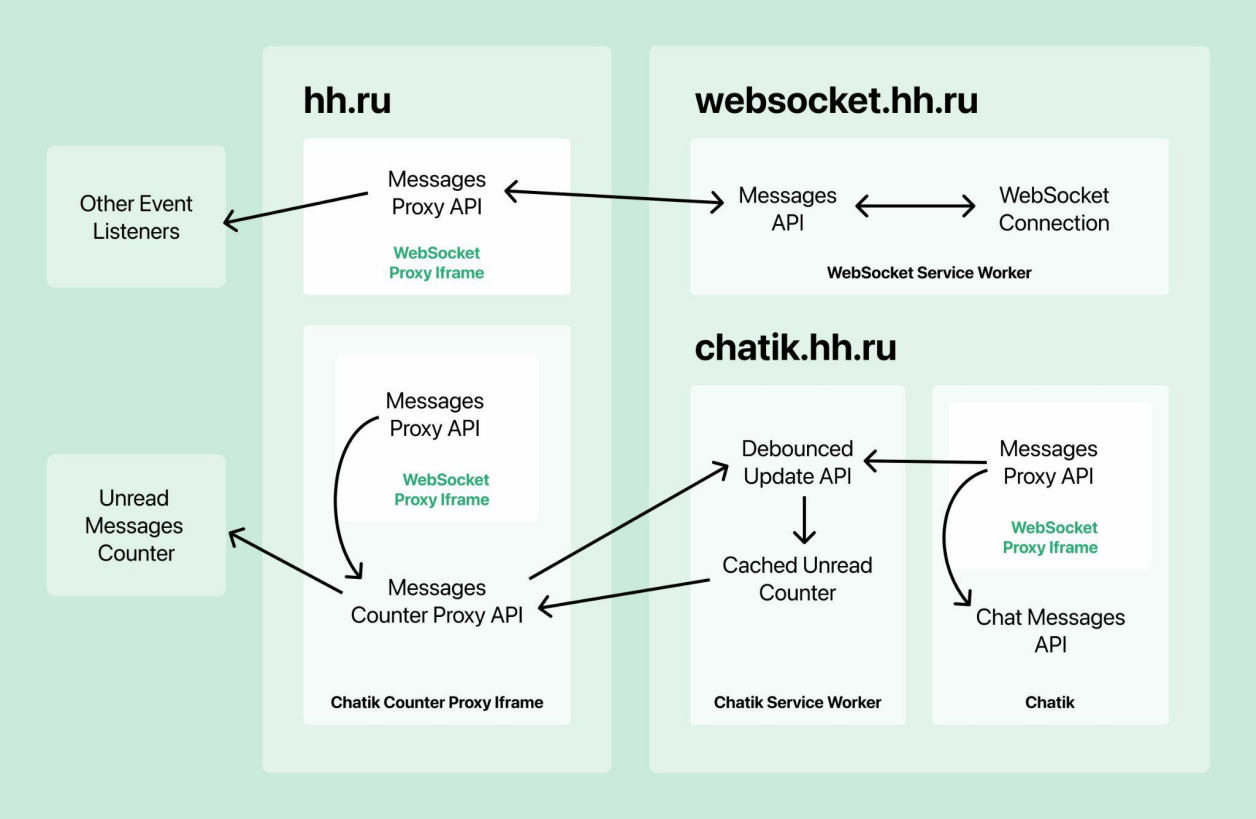
Живет даже после закрытия страницы

Может принимать и отображать push-уведомления

Пример из hh.ru



Пример из hh.ru



https://habr.com/ru/companies/hh/articles/652089/

Готов ответить на ваши вопросы

Домашнее задание

Реализовать drag’n’drop элементов с помощью курсора мыши, либо касанием пальца на тач-устройстве

- Зона создающая новый элемент при перетаскивании
- Элемент — квадрат 100x100px случайного цвета
- Область при перетаскивании в которую элементы располагаются в соответствии с сеткой
- Область сохраняющая положение перемещенного в неё элемента
- Если элемент опущен мимо вышеописанных областей — он исчезает

Здесь берем элемент

Тут размещаем по сетке

Тут размещаем свободно

Спасибо за внимание!

Желаю успехов в подготовке ДЗ